BACKPROPAGATION

Обратное распространение ошибки

Команда [10] · Расторгуев Михаил, Аюшева Рената

Постановка задачи

Сеть — функция с весами w

$$o = f(x, w)$$

Ошибка — расстояние до правильного ответа y

$$E = \frac{1}{2} \cdot (y - o)^2$$

Обучение — градиентный спуск, η — скорость обучения

$$w \leftarrow w - \eta \cdot \partial E / \partial w$$

Главный вопрос

Как найти $\partial E/\partial w$ для каждого веса сети — быстро и точно?

Цена одного шага обучения

Менять веса по одному и смотреть на ошибку

$\approx N$

прогонов сети на один шаг, где N — число весов

Обратное распространение ошибки

≈ 2

прохода: один прямой и один обратный, при любом N

В актуальном виде метод был предложен в 1986 году Дэвидом Э. Румельхартом, Джеффри Э. Хинтоном и Рональдом Дж. Вильямсом, а также независимо и одновременно красноярскими математиками С. И. Барцевым и В. А. Охониным.

Основная идея

Сеть — цепочка функций. По цепному правилу производная ошибки по весу собирается из производных вдоль пути от этого веса к ошибке.

Прямой проход: считаем и запоминаем

$$x \to h \to o \to E$$

Обратный проход: передаём ошибку назад

$$\partial E/\partial W \leftarrow d_h \leftarrow d_o \leftarrow E$$

d — «дельта» нейрона: насколько ошибка чувствительна к его входной сумме. Дельта слоя считается из дельт следующего слоя, поэтому ничего не вычисляется дважды.

Математический аппарат

Ошибка на одном примере	$E = \frac{1}{2} \cdot (y - o)^2$
Дельта выходного нейрона	$d_o = (o - y) \cdot o \cdot (1 - o)$
Дельта скрытого нейрона j	$d_h[j] = d_o \cdot V[j] \cdot h[j] \cdot (1 - h[j])$
Градиенты по весам	$\partial E / \partial V[j] = d_o \cdot h[j]$ $\partial E / \partial W[j][i] = d_h[j] \cdot x[i]$

Активация — сигмоида σ(z) = 1 / (1 + e^(−z)). Её производная σ′ = σ·(1 − σ), отсюда множители o·(1 − o) и h·(1 − h).

Алгоритм

1

Прямой проход

Подаём x, считаем h и o, запоминаем их

2

Ошибка выхода

Сравниваем o с ответом y, получаем d_o

3

Обратный проход

Передаём d_o назад через веса, получаем d_h

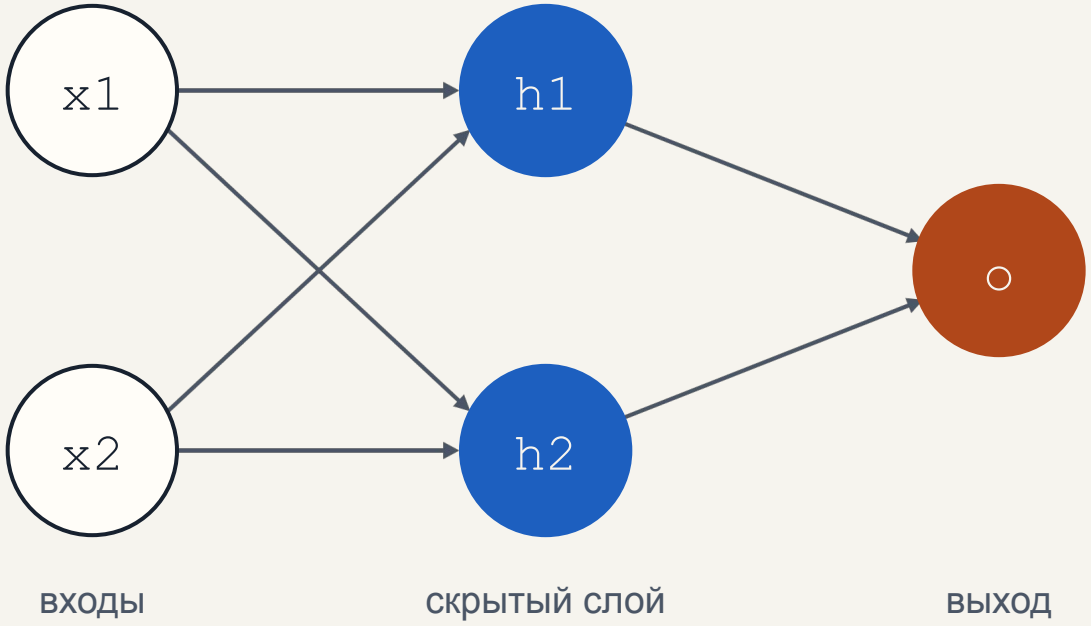
4

Обновление весов

Сдвигаем каждый вес против его градиента

Повторяем для каждого обучающего примера, эпоха за эпохой, пока ошибка не перестанет уменьшаться.

Численный пример: исходные данные



Что	Обозначение	Значение
Вход	x1, x2	1; 0.5
Правильный ответ	y	1
Веса в h1	W11, W12, b1	0.1; 0.2; 0.1
Веса в h2	W21, W22, b2	0.3; 0.4; 0.1
Веса в o	V1, V2, c	0.5; 0.6; 0.2
Скорость обучения	η	0.5

Все нейроны — сигмоида, ошибка $E = \frac{1}{2} \cdot (y - o)^2$.

Шаг 1. Прямой проход

Величина	Вычисление	Значение
z1	0.1·1 + 0.2·0.5 + 0.1	0.3
h1	σ(0.3)	0.5744
z2	0.3·1 + 0.4·0.5 + 0.1	0.6
h2	σ(0.6)	0.6457
z_o	0.5·0.5744 + 0.6·0.6457 + 0.2	0.8746
o	σ(0.8746)	0.7057
E	½·(1 − 0.7057)²	0.0433

Сеть ответила 0.7057 вместо 1. Значения h1, h2 и o запоминаем: они понадобятся на обратном проходе.

Шаг 2. Обратный проход

Величина	Вычисление	Значение
d_o	(0.7057 − 1)·0.7057·(1 − 0.7057)	−0.0611
∂E/∂V1	d_o·h1 = −0.0611·0.5744	−0.0351
∂E/∂V2	d_o·h2 = −0.0611·0.6457	−0.0395
d_h1	d_o·V1·h1·(1 − h1) = −0.0611·0.5·0.2445	−0.0075
d_h2	d_o·V2·h2·(1 − h2) = −0.0611·0.6·0.2288	−0.0084
∂E/∂W11	d_h1·x1 = −0.0075·1	−0.0075
∂E/∂W12	d_h1·x2 = −0.0075·0.5	−0.0037
∂E/∂W21	d_h2·x1 = −0.0084·1	−0.0084
∂E/∂W22	d_h2·x2 = −0.0084·0.5	−0.0042

Дельты выделены цветом. Градиенты по смещениям: ∂E/∂c = d_o, ∂E/∂b1 = d_h1, ∂E/∂b2 = d_h2

Шаг 3. Обновление весов

Вес	Было	Градиент	Стало
V1	0.5	−0.0351	0.5176
V2	0.6	−0.0395	0.6197
c	0.2	−0.0611	0.2306
W11	0.1	−0.0075	0.1037
W12	0.2	−0.0037	0.2019
W21	0.3	−0.0084	0.3042
W22	0.4	−0.0042	0.4021
b1	0.1	−0.0075	0.1037
b2	0.1	−0.0084	0.1042

Ошибка E после одного шага

0.0433 → 0.0400

Шагов	Выход o	Ошибка E
1	0.7172	0.0400
10	0.7888	0.0223
100	0.9235	0.0029
1000	0.9786	0.0002

Правило: стало = было − 0.5 · градиент. Проверка: численная производная по V1 даёт те же −0.0351.

Python: сеть и прямой проход

```
import math, random

def sigmoid(z):
    return 1 / (1 + math.exp(-z))

class Network: # сеть 2-2-1
    def __init__(self, n_in=2, n_hid=2, seed=1):
        rnd = random.Random(seed)
        self.W = [[rnd.uniform(-1, 1) for _ in range(n_in)] for _ in range(n_hid)]
        self.b = [rnd.uniform(-1, 1) for _ in range(n_hid)]
        self.V = [rnd.uniform(-1, 1) for _ in range(n_hid)]
        self.c = rnd.uniform(-1, 1)

    def forward(self, x): # запоминаем h и o
        self.h = [sigmoid(sum(w * xi for w, xi in zip(row, x)) + bj)
                  for row, bj in zip(self.W, self.b)]
        self.o = sigmoid(sum(v * hj for v, hj in zip(self.V, self.h)) + self.c)
        return self.o
```

Python: обратный проход

```
def backward(self, x, y, lr):
    # ошибка выходного нейрона: dE/dz_o
    d_o = (self.o - y) * self.o * (1 - self.o)
    # ошибки скрытых нейронов: d_o идёт назад через веса V
    d_h = [d_o * v * hj * (1 - hj) for v, hj in zip(self.V, self.h)]
    # шаг градиентного спуска
    for j, hj in enumerate(self.h):
        self.V[j] -= lr * d_o * hj
        self.b[j] -= lr * d_h[j]
        for i, xi in enumerate(x):
            self.W[j][i] -= lr * d_h[j] * xi
    self.c -= lr * d_o
```

Весь обратный проход — две строки: **d_o** и **d_h**. Это формулы со слайда «Математический аппарат».

Остальное — обновление: градиент равен дельте нейрона, умноженной на пришедший по весу сигнал.

Python: обучение и результат

```
    def train(self, data, epochs, lr):
        for epoch in range(epochs):
            for x, y in data:
                self.forward(x)
                self.backward(x, y, lr)

XOR = [([0, 0], 0), ([0, 1], 1),
       ([1, 0], 1), ([1, 1], 0)]
net = Network()
net.train(XOR, epochs=10001, lr=0.5)
for x, y in XOR:
    print(x, round(net.forward(x), 3))
```

Суммарная ошибка за эпоху

0.5389 → 0.0006

Вход	Сеть	Нужно
[0, 0]	0.018	0
[0, 1]	0.984	1
[1, 0]	0.984	1
[1, 1]	0.017	0

Задача XOR: ответ 1, когда входы разные. Сеть без скрытого слоя её не решает, а эта — выучила за 10 000 эпох.

Полный код с печатью ошибки — в файле backprop.py

Альтернативы

Метод	Как находит направление	Цена одного шага
Обратное распространение	Точный градиент по цепному правилу, от выхода ко входу	≈ 2 прохода сети
Численное дифференцирование	Сдвигаем каждый вес на ε и смотрим на изменение ошибки	2N проходов
Прямой режим автодифференцирования	Точные производные, но по одному весу за проход	N проходов
Генетические алгоритмы, эволюционные стратегии	Случайные изменения весов и отбор лучших, без градиента	Проход на каждую особь
Feedback alignment, Forward-Forward	Приближённый сигнал ошибки без точного обратного прохода	≈ 2 прохода

N — число весов сети

Сравнение на замере

Один и тот же градиент считаем обратным распространением и численным дифференцированием. Во сколько раз численный способ медленнее:

110 весов

×90

930 весов

×1000

3050 весов

×3000

Разрыв растёт вместе с числом весов. Сами значения градиентов совпали: расхождение меньше 1e-10.

Наш замер: сеть с одним скрытым слоем, чистый Python, округлено. Точные времена зависят от компьютера

Ограничения метода

Дифференцируемость

У функций активации и ошибки должна быть производная. Здесь выигрывают методы без градиента.

Затухание градиентов

В глубоких сетях дельты перемножаются слой за слоем и могут стать слишком малыми или слишком большими.

Локальные минимумы

Глобальный минимум не гарантирован: результат зависит от начальных весов и скорости обучения.

Память

Выходы всех слоёв нужно хранить до обратного прохода.

Заключение

1. Обратное распространение считает градиент ошибки по всем весам сети за один прямой и один обратный проход.
2. В основе — цепное правило: дельта каждого слоя вычисляется из дельт следующего.
3. Метод реализуется «с нуля» в несколько десятков строк на чистом Python.
4. Альтернативы либо требуют порядка N проходов, либо сходятся медленнее. Они полезны для проверки градиентов и для недифференцируемых задач.
5. Ограничения: нужна дифференцируемость, градиенты могут затухать, глобальный минимум не гарантирован.

Источники

Rumelhart D. E., Hinton G. E., Williams R. J. Learning representations by back-propagating errors. Nature, 1986, vol. 323, pp. 533–536.

Werbos P. Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences. PhD thesis, Harvard University, 1974.

Linnainmaa S. Магистерская диссертация об обратном режиме автоматического дифференцирования. University of Helsinki, 1970.

Goodfellow I., Bengio Y., Courville A. Deep Learning. MIT Press, 2016. Раздел 6.5.

Nielsen M. Neural Networks and Deep Learning. 2015. neuralnetworksanddeeplearning.com

Николенко С., Кадурин А., Архангельская Е. Глубокое обучение. Погружение в мир нейронных сетей. СПб.: Питер, 2018.

Lillicrap T. et al. Random synaptic feedback weights support error backpropagation for deep learning. Nature Communications, 2016.

Hinton G. The Forward-Forward Algorithm: Some Preliminary Investigations. arXiv:2212.13345, 2022.

Salimans T. et al. Evolution Strategies as a Scalable Alternative to Reinforcement Learning. arXiv:1703.03864, 2017.